

Atlas GitOps Control Plane Design

Atlas Architecture Group

August 17, 2026

Contents

0.1	0. 文档职责与规范语义	2
0.2	1. GitOps Ownership & Trust Boundary	2
0.2.1	1.1 Git owns Definition	3
0.2.2	1.2 Bootstrap owns Instantiation	3
0.2.3	1.3 Argo CD owns GitOps Reconciliation	3
0.2.4	1.4 Operators own Runtime Behavior	4
0.3	2. Canonical AppProject Model	4
0.4	3. Three-Tier Trust Isolation	4
0.4.1	3.1 Tier-0 —__INLINE_CODE_0__	4
0.4.2	3.2 Tier-1 —__INLINE_CODE_0__	5
0.4.3	3.3 Tier-2 —__INLINE_CODE_0__	5
0.5	4. Bootstrap Trust Chain	6
0.6	5. External Root Anchor	6
0.6.1	5.1 Root Anchor Security Consequence	7
0.7	6. Two-Level Reconciliation DAG	7
0.8	7. Level 1 —Root Macro DAG	7
0.8.1	7.1 Root Macro Waves	8
0.9	8. Level 2 —Platform Capability DAG	8
0.10	9. Platform Capability Wave DAG	9
0.10.1	9.1 Wave -100 —Foundation	9
0.10.2	9.2 Wave -90 —Management Plane	10
0.10.3	9.3 Wave -50 —Operators	10
0.10.4	9.4 Wave -30 —Infrastructure Controllers	10
0.10.5	9.5 Wave -10 —Platform Services	11
0.11	10. Domain Ontology	11
0.11.1	10.1 __INLINE_CODE_0__	11
0.11.2	10.2 __INLINE_CODE_0__	11
0.11.3	10.3 __INLINE_CODE_0__	12
0.11.4	10.4 __INLINE_CODE_0__	12
0.11.5	10.5 __INLINE_CODE_0__	12
0.11.6	10.6 __INLINE_CODE_0__	13
0.11.7	10.7 __INLINE_CODE_0__	13
0.11.8	10.8 __INLINE_CODE_0__	13
0.12	11. Leaf Application Contract	13
0.13	12. Application Health Gate Contract	14
0.13.1	12.1 Application Health Capability	14
0.13.2	12.2 Child Automated Sync	14
0.13.3	12.3 Macro Health Gate	15
0.14	13. Workload Control Model	15

0.14.1	13.1 Workload Control Application	16
0.14.2	13.2 Tenant Leaf Applications	16
0.14.3	13.3 Why	16
0.15	14. Workload Directory	17
0.16	15. Data Bootstrap Jobs	17
0.16.1	15.1 Safe Initialization	17
0.16.2	15.2 Destructive Migration	18
0.17	16. Sync Wave Scope	18
0.18	17. Environment Hydration	18
0.18.1	17.1 Root Hydration	19
0.18.2	17.2 Platform Hydration	19
0.18.3	17.3 Environment-specific K8s State	19
0.19	18. Bootstrap Adoption Contract	20
0.19.1	18.1 __INLINE_CODE_0__	20
0.19.2	18.2 Recovery Seed	20
0.20	19. Dual Deletion Protection	20
0.20.1	19.1 Parent-level Prune Protection	20
0.20.2	19.2 No Cascading Finalizer	21
0.20.3	19.3 Protection Boundary	21
0.21	20. Secret Dependency Contract	21
0.22	21. Emergency Recovery & Blast Radius Freeze	22
0.22.1	21.1 Detect	22
0.22.2	21.2 Freeze Outside-In	22
0.22.3	21.3 Snapshot	23
0.22.4	21.4 Restore Seed	23
0.22.5	21.5 Repair Git	23
0.22.6	21.6 Resume Inside-Out	23
0.23	22. ApplicationSet Boundary	24
0.24	23. AI & Human Control Boundary	24
0.25	24. Phase 2 Governance Roadmap	25
0.25.1	Phase 2A —Safety Foundation	25
0.25.2	Phase 2B —Tenant Platform	25
0.25.3	Phase 2C —Developer Experience	25
0.25.4	Phase 2D —Agent Automation	26
0.26	25. GitOps Control Plane Invariants	26
0.26.1	GITOPS-01	26
0.26.2	GITOPS-02	26
0.26.3	GITOPS-03	26
0.26.4	GITOPS-04	26
0.26.5	GITOPS-05	26
0.26.6	GITOPS-06	26
0.26.7	GITOPS-07	26
0.26.8	GITOPS-08	27
0.26.9	GITOPS-09	27
0.26.10	GITOPS-10	27
0.26.11	GITOPS-11	27
0.26.12	GITOPS-12	27
0.26.13	GITOPS-13	27
0.26.14	GITOPS-14	27
0.27	26. Final Frozen Control Graph	27
0.28	27. Architecture Freeze Statement	28

Version: v1.0.3
Status: Frozen Production Baseline (Two-Level DAG Architecture Freeze Point)
Domain: GitOps Control Plane / Platform Engineering

0.1 0. 文档职责与规范语义

本文档是：

Atlas Architecture Design v1.0.2

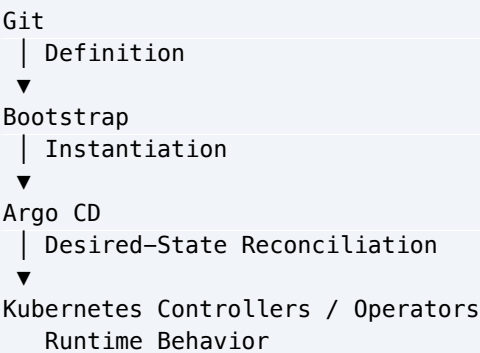
在 GitOps / Argo CD 领域中的规范性执行设计。
本文档负责定义：

- External Root Anchor;
- AppProject Trust Chain;
- Root Macro DAG;
- Platform Capability DAG;
- Workload Control Model;
- Sync Wave;
- Application Health Gate;
- Environment Hydration;
- Child Automated Sync;
- Deletion Protection;
- Bootstrap Adoption;
- Break-Glass Recovery。

本文不得改变 Architecture Design 中定义的平台级不变量。

0.2 1. GitOps Ownership & Trust Boundary

Atlas GitOps 控制面采用四阶控制权模型。



0.2.1 1.1 Git owns Definition

Git 保存：

- Root Anchor Template；
- Application Graph；
- AppProject；
- Platform Desired State；
- Workload Desired State；
- Policy；
- Overlay；
- Version Lock。

Git 是 Desired State Definition Authority。

0.2.2 1.2 Bootstrap owns Instantiation

Bootstrap 负责：

1. 安装最小 Argo CD Seed；
2. 创建 __INLINE_CODE_0__；
3. 注入与 Full Desired State 相同的 Application Health Capability；
4. 根据环境上下文渲染 Root Anchor；
5. 将 External Root Anchor 注入 Kubernetes。

Bootstrap 完成后退出。

0.2.3 1.3 Argo CD owns GitOps Reconciliation

Argo CD 对所有进入 GitOps Domain 的资源负责：

- Sync；
- Drift Detection；
- Reconciliation；
- Health；
- Prune。

External Root Anchor 本身除外。

External Root Anchor 不存在 Parent Application，因此不属于 parent-driven steady-state reconciliation。

0.2.4 1.4 Operators own Runtime Behavior

例如：

```
Argo CD
↓
FlinkDeployment
↓
Flink Operator
↓
Runtime
```

Argo CD 不尝试取代 Domain Operator。

0.3 2. Canonical AppProject Model

Atlas 正式冻结三个 Canonical AppProject Identifier。

```
atlas-bootstrap
platform-project
workload-project
```

任何实现、图示、Manifest 与 Conformance Audit MUST 使用上述准确名称。

禁止使用以下模糊别名代替实际资源名称：

```
platform
workload
bootstrap-project
```

0.4 3. Three-Tier Trust Isolation

0.4.1 3.1 Tier-0 — __INLINE_CODE_0__

__INLINE_CODE_0__ 由 Bootstrap 命令式创建。

其唯一职责是承载完成 Trust Chain 自举所必需的 Application。

允许：

- External Root Anchor；
- Project Bootstrap。

禁止：

- 普通 Platform Application；
- Workload Application；
- Developer Workload；
- 日常平台组件。

__INLINE_CODE_0__ 是 Bootstrap Capability Sandbox，不是普通租户 Project。

0.4.2 3.2 Tier-1 —__INLINE_CODE_0__

由 Project Bootstrap 声明式创建。
负责：

- Platform Control；
- Argo CD Self-Management；
- Infrastructure；
- Governance；
- Operator；
- Controller；
- Workload Orchestrator；
- 需要高权限的管理组件。

权限必须采用显式 allow-list，而不是无约束开放。

0.4.3 3.3 Tier-2 —__INLINE_CODE_0__

由 Project Bootstrap 声明式创建。
用于最终业务工作负载。
其原则是：

Restricted by default.

必须禁止或严格限制：

- CRD；
- ClusterRole；
- ClusterRoleBinding；
- Cluster-wide privileged resources；
- 任意 Platform Namespace；
- Argo CD Control Namespace。

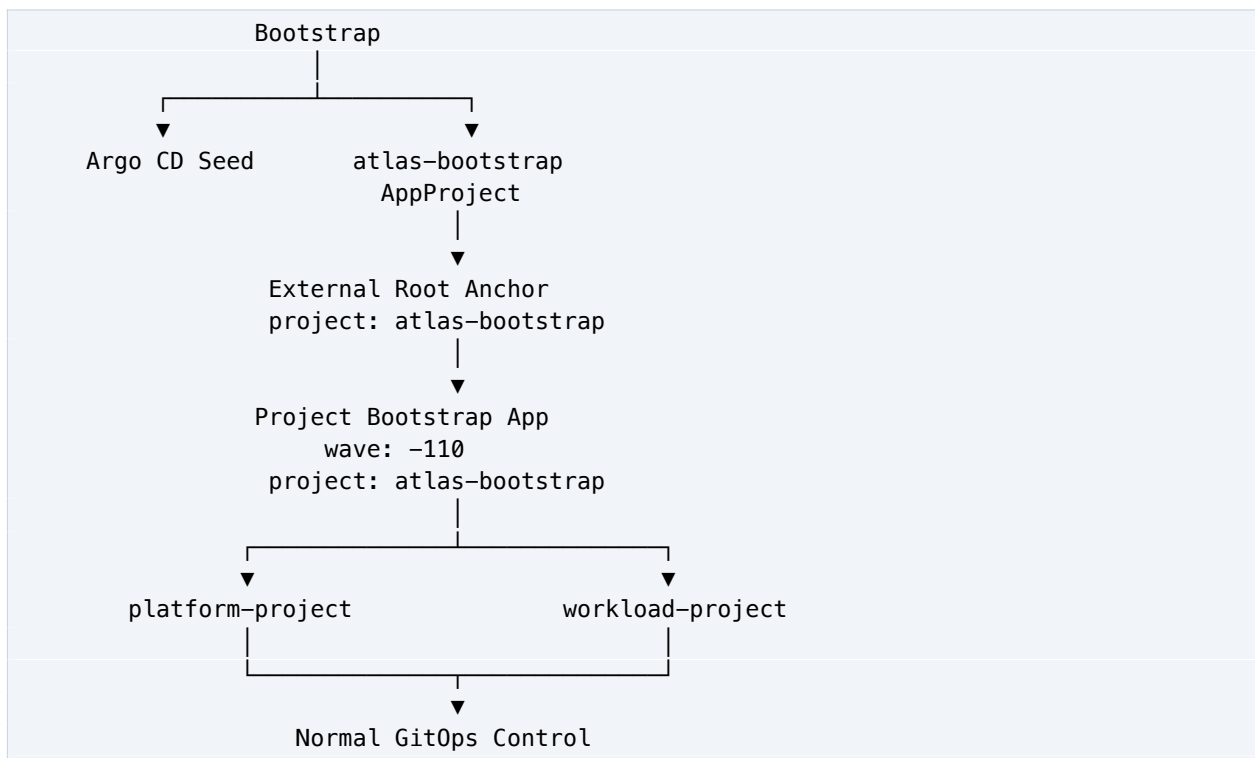
Namespaced：

- ServiceAccount；
- Role；
- RoleBinding；

按平台策略受控开放。

0.5 4. Bootstrap Trust Chain

Atlas 的完整无环 Bootstrap Chain 为：



不存在：

```

Application requires Project
      ↑
Project requires same Application
  
```

的循环依赖。

0.6 5. External Root Anchor

External Root Anchor 是整个 GitOps Control Graph 的外部根节点。

其模板位于：

```
bootstrap/argocd/root-app.yaml.tpl
```

Bootstrap 根据：

```
env/
```

中的环境上下文进行渲染，并将实例写入：

```
.state/root-app.yaml
```

随后注入 Kubernetes。

其 Desired Definition 来源于 Git，但该活体对象不存在 Parent Application。

因此：

Git owns its contract, Bootstrap owns its instantiation, but no parent Application owns its reconciliation.

这是有意设计，不属于配置缺陷。

0.6.1 5.1 Root Anchor Security Consequence

由于 Root Anchor 不被父级 Argo CD Application 修复：

```
kubectl edit Application <root>
```

理论上可以导致：

```
Live Root
≠
Git Template
```

因此 External Root Anchor 必须通过其他控制机制保护：

- Kubernetes RBAC；
- Argo RBAC；
- Admission Policy；
- Audit；
- Tier-0 Human Judgment；
- Drift Detection。

未来 Kyverno / 等价准入治理应成为该 Trust Boundary 的重要补强。

0.7 6. Two-Level Reconciliation DAG

Atlas v1.0.3 正式放弃：

Flat Global Root Component DAG

同时禁止：

Filesystem-shaped Deep Nested DAG.

正式模型为：

Minimal Root Macro DAG + Flat Capability DAG

0.8 7. Level 1 —Root Macro DAG

__INLINE_CODE_0__ 只负责 Trust Transition。

标准结构：

```
gitops/root/
├── base/
│   ├── kustomization.yaml
│   ├── project-bootstrap-app.yaml
│   ├── platform-control-app.yaml
│   └── workload-control-app.yaml
└── overlays/
    └── local-orbstack/
```

```

├── kustomization.yaml
├── prod/
│   └── kustomization.yaml

```

Root SHALL NOT 直接列举：

```

flink
envoy
prometheus
redpanda
minio
loki
tempo
...

```

0.8.1 7.1 Root Macro Waves

Root Synchronization Scope 定义：

```

-110 Project Bootstrap
-100 Platform Control
   0 Workload Control

```

控制流程：

```

Project Bootstrap Healthy
      ↓
Platform Control starts
      ↓
Platform Control Healthy
      ↓
Workload Control starts

```

Root 的 Wave 与 Platform Control 内部 Wave 属于不同 Sync Scope。
数字相同不表示同一全局时间点。

0.9 8. Level 2 —Platform Capability DAG

Platform Control Application 指向：

```
gitops/platform/applications/overlays/<environment>/
```

例如：

```

gitops/platform/applications/
├── base/
│   ├── kustomization.yaml
│   ├── foundation-app.yaml
│   ├── argocd-self-app.yaml
│   ├── sealed-secrets-app.yaml
│   ├── cert-manager-app.yaml
│   ├── flink-operator-app.yaml
│   ├── envoy-gateway-app.yaml
│   ├── kube-prometheus-stack-app.yaml
│   ├── redpanda-app.yaml
│   └── minio-app.yaml

```

```
└─ overlays/
   └─ local-orbstack/
      └─ prod/
```

这里保存的是：

Application Graph Metadata

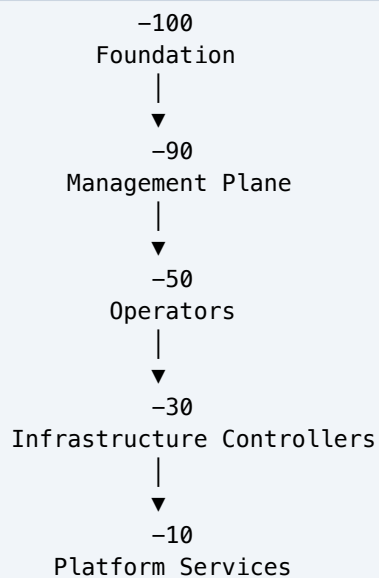
而不是组件实现。

0.10 9. Platform Capability Wave DAG

Platform Control Synchronization Scope 内采用：

```
-100 Foundation
-90 Management Plane
-50 Operators
-30 Infrastructure Controllers
-10 Platform Services
```

逻辑如下：



0.10.1 9.1 Wave -100 —Foundation

包括：

- Namespaces；
- ResourceQuotas；
- LimitRanges；
- 其他 Kubernetes 原生逻辑边界。

环境强绑定资源必须通过 Environment Hydration 进入。

0.10.2 9.2 Wave -90 —Management Plane

包括：

- `__INLINE_CODE_0__`；
- Sealed Secrets；
- cert-manager；
- OIDC；
- Policy Engine；
- ESO；
- 其他平台自身管理与治理能力。

其中 Secret Management Controller 必须早于其 Consumer。

0.10.3 9.3 Wave -50 —Operators

包括：

- Flink Kubernetes Operator；
- Spark Operator；
- 其他扩展 Kubernetes API 的 Domain Operator。

必须在对应 CR 出现前确保：

- CRD 已注册；
 - Controller Ready；
 - Webhook Ready；
 - Application Healthy。
-

0.10.4 9.4 Wave -30 —Infrastructure Controllers

包括：

- Envoy Gateway Controller；
- Prometheus Operator；
- 其他底层 Controller。

“Controller”是依赖角色，不是目录领域。

例如：

Envoy Gateway

仍然属于：

platform/networking/

而不是：

platform/controllers/

0.10.5 9.5 Wave -10 —Platform Services

包括：

- Redpanda / Kafka；
- MinIO；
- Database；
- 其他 Stateful Platform Services。

“Platform Service”同样是 DAG Role，不是 Domain Directory。

0.11 10. Domain Ontology

目录只表达 Domain Ownership。

```
gitops/platform/  
├─ applications/  
├─ foundation/  
├─ management/  
├─ operators/  
├─ networking/  
├─ observability/  
├─ messaging/  
└─ storage/
```

0.11.1 10.1 __INLINE_CODE_0__

不是业务领域。

仅保存：

- Application；
 - Kustomization；
 - Application DAG Overlay；
 - Wave Metadata；
 - Parent-level protection metadata。
-

0.11.2 10.2 __INLINE_CODE_0__

Kubernetes 原生逻辑边界。

0.11.3 10.3 __INLINE_CODE_0__

平台自身治理能力：

- Argo CD Self Management；
- AppProjects；
- RBAC；
- OIDC；
- Sealed Secrets；
- ESO；
- cert-manager；
- Policy Engine。

AppProject Desired State 必须归属：

```
platform/management/projects/
```

Project Bootstrap Application 只是使用 __INLINE_CODE_0__ 去同步这些对象。
因此：

```
Bootstrap lifecycle ≠ AppProject domain ownership.
```

0.11.4 10.4 __INLINE_CODE_0__

领域扩展 API Controller：

- Flink Operator；
- Spark Operator；
- 其他平台领域 Operator。

0.11.5 10.5 __INLINE_CODE_0__

网络与流量能力：

- Envoy Gateway；
- Gateway API；
- Gateway；
- HTTPRoute；
- NetworkPolicy-related resources。

0.11.6 10.6 __INLINE_CODE_0__

统一遥测：

- Kube-Prometheus-Stack；
 - Alloy；
 - OpenTelemetry Collector；
 - Loki；
 - Tempo；
 - OTLP Transport。
-

0.11.7 10.7 __INLINE_CODE_0__

异步事件总线：

- Redpanda；
 - Kafka。
-

0.11.8 10.8 __INLINE_CODE_0__

数据持久化平台：

- MinIO；
 - 未来其他存储服务。
-

0.12 11. Leaf Application Contract

每个可独立：

- 失败；
- 暂停；
- 升级；
- 回滚；
- 观察；
- 恢复；

的平台能力 SHOULD 成为独立 Leaf Application。

Leaf Application 是：

Minimum Reconciliation and Failure Isolation Unit.

禁止仅因为：

```
platform/observability/
```

存在子目录，就形成：

```
platform-app
↓
observability-app
↓
prometheus-app
```

这种目录镜像式 Application 链。

默认最大结构为：

```
External Root
↓
Platform Control
↓
Leaf Application
↓
Resources
```

任何额外 Application 层 MUST 由 ADR 证明其具有真实控制边界价值。

0.13 12. Application Health Gate Contract

Sync Wave 只有结合 Health Gate 才构成真正的依赖图。

Atlas 的 DAG Contract 是：

```
Sync Wave
+
Child Automated Sync
+
Application Health
=
Deterministic Application DAG
```

三者缺一不可。

0.13.1 12.1 Application Health Capability

Bootstrap Seed 与 __INLINE_CODE_0__ 必须注入同一份：

```
argoproj.io/Application
```

健康评估能力。

该 Capability MUST 来自同一份受版本控制资产。

禁止：

```
Bootstrap health semantics
≠
Steady-state health semantics
```

0.13.2 12.2 Child Automated Sync

所有作为 DAG 前置条件的 Child Application：

MUST enable Automated Sync

或者使用一个外部确定性触发机制。
禁止出现：

```
Wave -50 App
Healthy only after manual sync
```

却被用作：

```
Wave -30 prerequisite
```

否则 DAG 会永久阻塞。

0.13.3 12.3 Macro Health Gate

Root 不需要理解 Platform 内部组件。

Root 只判断：

```
platform-control = Healthy
```

因此：

```
Foundation
+
Management
+
Operators
+
Controllers
+
Platform Services
  ↓
Converged
  ↓
Platform Control Healthy
  ↓
Workload Control may proceed
```

这将：

Platform Ready

从人类经验判断转化为机器可判定状态。

0.14 13. Workload Control Model

Workload Control 必须区分：

Control-plane Orchestrator

与：

Tenant Payload Application.

这是 Tier-1 / Tier-2 权限隔离的核心。

0.14.1 13.1 Workload Control Application

Root 中:

```
workload-control-app.yaml
```

如果其职责是创建:

- Application;
- ApplicationSet;
- 其他 Argo CD Control Objects;

则其本质属于:

```
Tier-1 Control Plane.
```

因此:

```
project: platform-project
```

而不是:

```
project: workload-project
```

0.14.2 13.2 Tenant Leaf Applications

由 Workload Control 生成或管理的最终业务 Application:

```
project: workload-project
```

其资源只能进入被允许的 Workload Namespace。

控制关系:

```

Workload Control
project: platform-project
    |
    ▼
Application / ApplicationSet
    |
    ▼
Tenant Leaf Application
project: workload-project
    |
    ▼
Developer Namespace
  
```

0.14.3 13.3 Why

禁止让 __INLINE_CODE_0__ 为了创建 Argo CD Application/ApplicationSet 而获得:

- Argo CD Namespace 写权限;
- Platform CR 写权限;
- 高危 Cluster Capability。

原则是:

WHO creates tenant Applications □ Tier-1 WHAT tenant Applications may deploy □ Tier-2

0.15 14. Workload Directory

建议标准：

```
gitops/workloads/
├── applications/
│   ├── base/
│   └── overlays/
└── apps/
    ├── clickstream-processor/
    ├── example-streaming-job/
    └── ...
```

其中：

applications/

保存 Workload Orchestration Metadata。

apps/

保存真正的业务 Desired State。

Phase 2B 可以将：

applications/

演进为 ApplicationSet Tenant Model。

0.16 15. Data Bootstrap Jobs

数据初始化逻辑不得因为某个 Wave 到达就默认具有破坏性执行权。

安全初始化必须：

100% Idempotent.

例如：

- create-if-not-exists；
- schema initialization；
- non-destructive seed。

0.16.1 15.1 Safe Initialization

允许进入自动 DAG。

推荐作为 Workload Control Scope 中：

+10

级别的初始化单元。

0.16.2 15.2 Destructive Migration

例如：

- DROP；
- destructive schema rewrite；
- irreversible state conversion；
- mass data deletion；

不得依赖普通 AutoSync 自动执行。
必须转化为：

Human-gated Workflow.

0.17 16. Sync Wave Scope

Atlas 正式规定：

Sync Wave is parent synchronization-scope local.

因此：

Root：

```
-110 Project Bootstrap
-100 Platform Control
  0 Workload Control
```

与 Platform Control：

```
-100 Foundation
  -90 Management
    -50 Operators
      -30 Controllers
        -10 Services
```

虽然存在相同数字：

```
-100
```

但它们不属于同一个全局调度空间。

禁止将所有 Wave 理解成：

Cluster-wide absolute sequence number.

0.18 17. Environment Hydration

Environment-specific state 与通用 Platform Domain 必须分离。

标准结构：

```

gitops/
├── root/
│   ├── overlays/
│   │   ├── local-orbstack/
│   │   └── prod/
│   └── environments/
│       ├── local-orbstack/
│       └── prod/
└── platform/
    ├── applications/
    │   ├── overlays/
    │   │   ├── local-orbstack/
    │   │   └── prod/

```

0.18.1 17.1 Root Hydration

Bootstrap 选择:

```
root/overlays/<environment>
```

作为 External Root Source。

0.18.2 17.2 Platform Hydration

__INLINE_CODE_0__ 选择:

```
platform/applications/overlays/<environment>
```

作为 Platform DAG。

0.18.3 17.3 Environment-specific K8s State

例如:

- local StorageClass;
- Cloud-specific CSI;
- environment-specific Gateway address;
- substrate-specific configuration;

必须位于:

```
gitops/environments/<environment>/
```

或由对应 Overlay 显式引用。

不得污染通用 Platform Domain。

0.19 18. Bootstrap Adoption Contract

Bootstrap 恢复必须满足：

```
Object Identity Parity
+
Version Parity
+
Health Capability Parity
+
CRD Compatibility
=
Bootstrap Adoption Contract
```

0.19.1 18.1 __INLINE_CODE_0__

Bootstrap 与 GitOps Self-management 必须共同消费：

```
versions.lock
```

禁止：

```
bootstrap Argo version
≠
argocd-self desired version
```

所导致的隐式：

- downgrade;
- upgrade;
- CRD incompatibility。

0.19.2 18.2 Recovery Seed

Break-Glass Seed 的目标是：

```
Minimal but adoption-compatible.
```

不是重新建立另一套独立控制平面。

0.20 19. Dual Deletion Protection

Atlas 实施两层独立保护。

0.20.1 19.1 Parent-level Prune Protection

所有由 Parent Application 管理的 Tier-0 / Tier-1 Child Application CR 应具有：

```
argocd.argoproj.io/sync-options: Prune=confirm
```

保护链：

```

Git accidentally removes child Application
      ↓
Parent attempts prune
      ↓
Prune suspended
      ↓
Human confirmation
  
```

0.20.2 19.2 No Cascading Finalizer

核心 Tier-0 / Tier-1 Application 默认不得携带级联资源删除 finalizer。
因此：

```

delete Application
      ↓
Application CR deleted
      ↓
Managed resources survive
  
```

0.20.3 19.3 Protection Boundary

上述保护主要针对：

Application Control Graph.

并不意味着：

```

StatefulSet
PVC
Kafka Cluster CR
Database
Namespace
  
```

自动获得相同保护。

对于高价值 Leaf Resources，必须根据风险显式配置：

- resource-level `__INLINE_CODE_0__`；
- `__INLINE_CODE_0__`；
- Admission Policy；
- Domain-specific retention mechanism。

0.21 20. Secret Dependency Contract

Phase 1 使用 Sealed Secrets 时：

```

Sealed Secrets Controller
      ↓
Controller Healthy
      ↓
SealedSecret Reconciliation
  
```

```

      ↓
Secret Exists
      ↓
Consumer Application
  
```

Consumer 不得在 Secret Capability Ready 前进入强依赖路径。
Phase 2A 切换 ESO 时同样遵守：

```

ESO
  ↓
External Secret Materialization
  ↓
Consumer
  
```

0.22 21. Emergency Recovery & Blast Radius Freeze

Atlas 使用：

Freeze Outside-In, Resume Inside-Out.

0.22.1 21.1 Detect

确认：

- 错误 Revision；
- 受影响 Application；
- 数据面状态；
- 爆炸半径；
- 是否仍有危险 AutoSync。

0.22.2 21.2 Freeze Outside-In

首先冻结：

External Root Anchor

然后依次关闭受影响 Control Application 的 AutoSync。

重点包括：

- Platform Control；
- argocd-self；
- 受影响 Leaf Applications；
- Workload Control。

目标不是删除资源，而是：

Stop propagation.

0.22.3 21.3 Snapshot

备份：

- Application 状态；
- AppProject 状态；
- 当前 Revision；
- Controller Configuration；
- 关键 Event / Audit Evidence。

0.22.4 21.4 Restore Seed

使用 Bootstrap 恢复满足 Adoption Contract 的 Argo CD Seed。

0.22.5 21.5 Repair Git

Git 必须回到：

Known-Good Revision.

不得依赖长期 Live Edit 作为恢复终态。

0.22.6 21.6 Resume Inside-Out

推荐顺序：

```
Argo Seed
↓
argocd-self
↓
Management Capability
↓
Platform Capability DAG
↓
Platform Control Healthy
↓
Workload Control
↓
Tenant Workloads
↓
External Root normal reconciliation
```

必须逐级验证 Health。

0.23 22. ApplicationSet Boundary

ApplicationSet 是 Atlas Phase 2B Tenant Platform 的核心候选机制，但不作为 v1.0.3 平台 Bootstrap Correctness 的必要基础。

当前 Frozen Baseline 的平台启动正确性依赖：

```
Application
+
App-of-Apps
+
Sync Wave
+
Application Health
```

ApplicationSet 的主要目标为：

- Tenant Expansion；
- Multi-cluster Workload Generation；
- Developer Self-Service；
- Fleet-style Application Generation。

ApplicationSet 不应取代 Tier-0 Root Trust Model。

0.24 23. AI & Human Control Boundary

__INLINE_CODE_0__：

```
READ      allowed
ANALYZE   allowed
PROPOSE   allowed
GENERATE   allowed
MERGE     human-gated
APPLY     human-gated
```

Tier-1：

允许未来在 Policy Guardrail 成熟后开放更高程度 Agent Autonomy。

Tier-2：

是最适合 AI Agent 自治执行的主要区域。

治理演进顺序必须是：

```
Guardrail
↓
Auditability
↓
Policy Enforcement
↓
Agent Autonomy
```

而不是先开放自治，再补安全机制。

0.25 24. Phase 2 Governance Roadmap

0.25.1 Phase 2A —Safety Foundation

引入：

- Kyverno / equivalent policy engine；
 - NetworkPolicy baseline；
 - ESO；
 - Root Anchor admission protection；
 - stronger audit controls。
-

0.25.2 Phase 2B —Tenant Platform

建设：

- ApplicationSet；
 - Tenant Model；
 - Namespace lifecycle；
 - workload-project policy；
 - multi-tenant boundaries。
-

0.25.3 Phase 2C —Developer Experience

建设：

- Backstage；
 - Golden Path；
 - Templates；
 - Platform API；
 - developer self-service。
-

0.25.4 Phase 2D —Agent Automation

在 Guardrail 完成后，开放：

- Agent-generated PR;
- Policy-verified changes;
- limited auto-merge;
- bounded autonomous remediation;
- declarative platform operation。

Agent 自治权不得跨越 Tier-0 Human Judgment Boundary。

0.26 25. GitOps Control Plane Invariants

Atlas GitOps Control Plane v1.0.3 冻结以下领域不变量。

0.26.1 GITOPS-01

External Root Anchor MUST remain outside parent Application reconciliation.

0.26.2 GITOPS-02

Canonical AppProjects MUST be:

```
atlas-bootstrap  
platform-project  
workload-project
```

0.26.3 GITOPS-03

Root MUST contain only macro trust/readiness control Applications.

0.26.4 GITOPS-04

Platform components MUST be orchestrated by Platform Control rather than enumerated directly under Root.

0.26.5 GITOPS-05

Directory hierarchy MUST NOT be mechanically mirrored as Application hierarchy.

0.26.6 GITOPS-06

Leaf Application SHOULD be the minimum failure-isolation unit.

0.26.7 GITOPS-07

Sync Wave MUST be interpreted within its parent synchronization scope.

0.26.8 GITOPS-08

Any Child Application acting as a DAG prerequisite MUST have deterministic synchronization.

0.26.9 GITOPS-09

Bootstrap and argocd-self MUST share Application Health semantics.

0.26.10 GITOPS-10

Bootstrap and argocd-self MUST share `__INLINE_CODE_0__`.

0.26.11 GITOPS-11

Workload orchestrators creating Argo CD control objects MUST operate in Tier-1.

0.26.12 GITOPS-12

Tenant payload Applications MUST operate in `__INLINE_CODE_0__`.

0.26.13 GITOPS-13

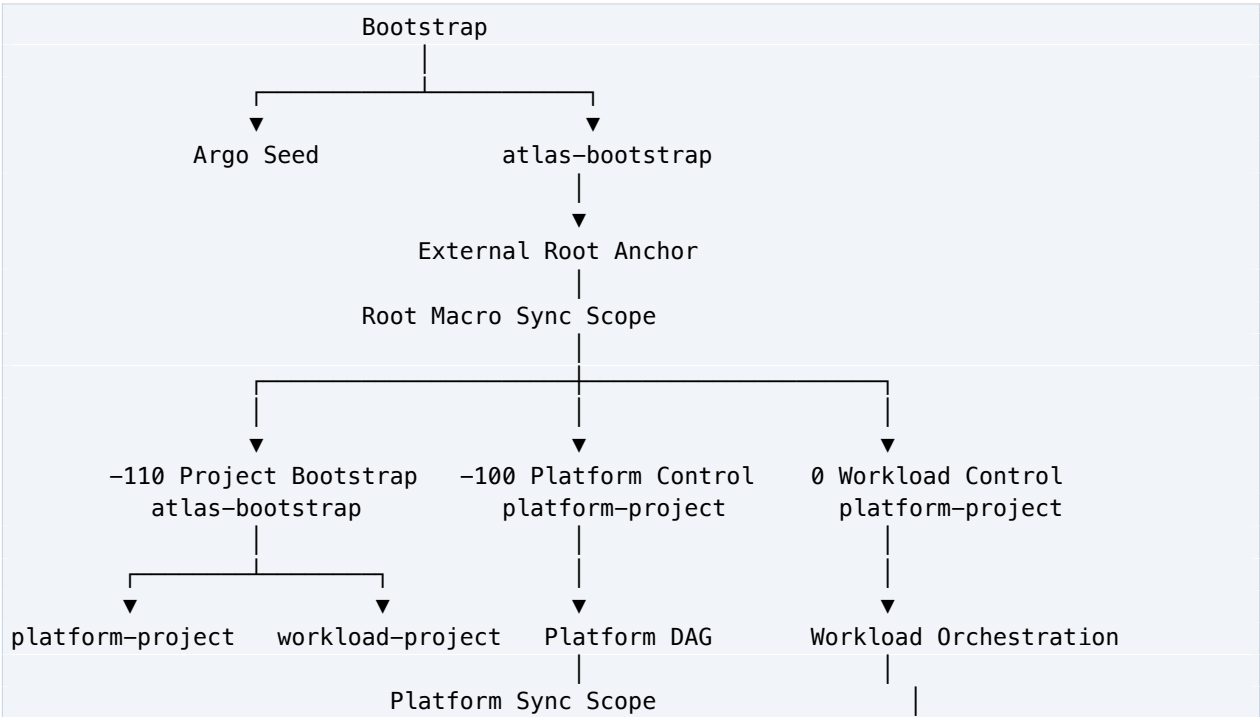
Tier-0 / Tier-1 control graph deletion MUST NOT implicitly destroy descendant runtime resources.

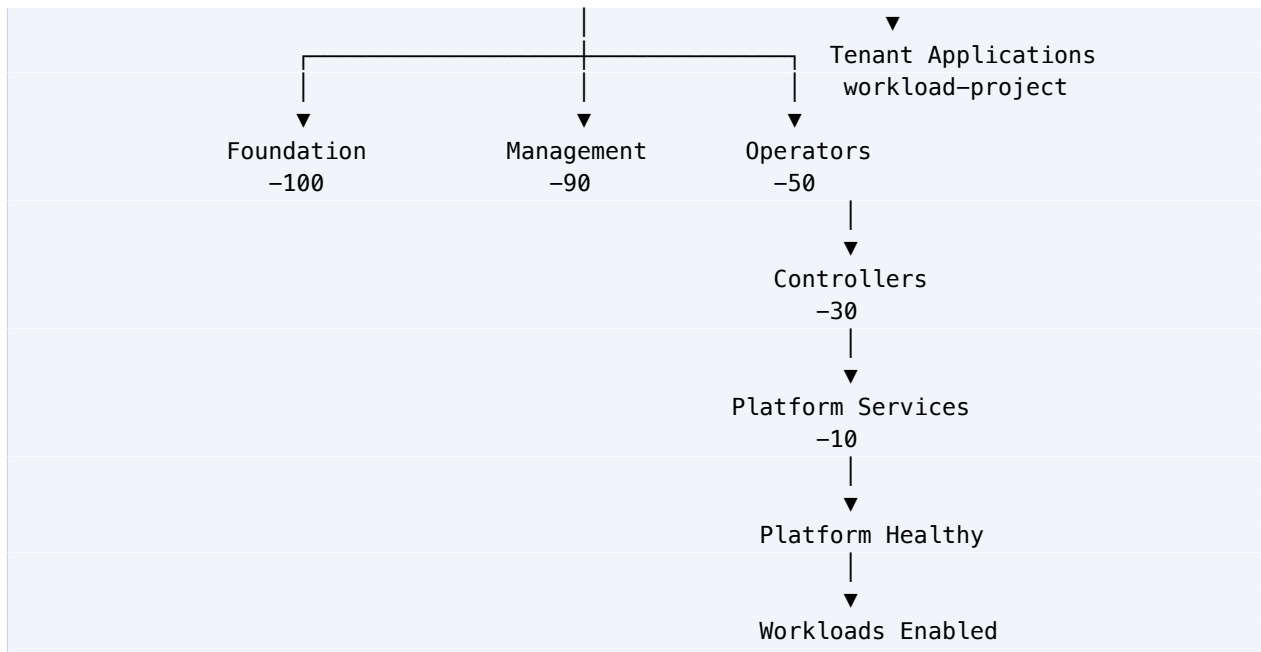
0.26.14 GITOPS-14

Destructive data mutation MUST NOT be triggered merely by automatic Wave progression.

0.27 26. Final Frozen Control Graph

Atlas v1.0.3 的最终规范拓扑为：





这张图是 Atlas GitOps Control Plane v1.0.3 的架构冻结点。

未来：

- 增加 Redpanda；
- 增加 ClickHouse；
- 增加 Spark；
- 增加 Loki；
- 增加 Tempo；
- 增加新的 Operator；

均应优先修改：

`gitops/platform/`

而不是：

`gitops/root/`

只有当：

- Trust Domain 改变；
- Control Ownership 改变；
- Macro Readiness Boundary 改变；
- Bootstrap Chain 改变；

时，才允许修改 Tier-0 Root Architecture。

0.28 27. Architecture Freeze Statement

Atlas GitOps v1.0.3 正式冻结以下模式：

- Minimal Root Macro DAG - Flat Platform Capability DAG - Tier-1 Workload Orchestration - Tier-2 Workload Execution
External Root Anchor
- **Minimal Root Macro DAG - Flat Platform Capability DAG - Tier-1 Workload Orchestration - Tier-2 Workload Execution**

其核心目标不是创建最复杂的 GitOps 拓扑，而是创建：

最容易证明、最容易恢复、最容易审计、最不容易被人类或 AI Agent 误操作的控制图。

从本版本开始，任何将 Root 重新扩张为 Platform Component Catalog，或仅为了匹配目录层次而引入深层 App-of-Apps 的实现，应默认视为对 Frozen Baseline 的偏离，并进入正式 Architecture Review。